

Progetto Basi di Dati 2024-25

"FANTASANREMO"

Parte III

32

Sarah Oliveri 5600355

Ana Maria Raducan 5797859

Andrea Pisano 6204123

1. PROGETTAZIONE FISICA

1A+1C - CARICO DI LAVORO

Q1 - QUERY CON SINGOLA SELEZIONE E NESSUN JOIN

LINGUAGGIO NATURALE

I 10 artisti più costosi (prezzo ≥ 50), ordinati dal più caro al meno caro e per ciascuno si restituisce CF, nome, cognome e prezzo.

SQL

```
EXPLAIN (ANALYZE, BUFFERS, VERBOSE)
SELECT cf, nome, cognome, prezzo
FROM artista
WHERE prezzo >= 50
ORDER BY prezzo DESC
LIMIT 10;
```

Q2 - QUERY CON CONDIZIONE DI SELEZIONE COMPLESSA E NESSUN JOIN

LINGUAGGIO NATURALE

I artisti con prezzo > 50 oppure con almeno 2 generi musicali escludendo quelli la cui provenienza è 'Roma'

SQL

```
EXPLAIN (ANALYZE, BUFFERS, VERBOSE)
SELECT cf, nome, cognome, prezzo, generi_musicali, provenienza
FROM artista
WHERE
(
    prezzo > 50
    OR array_length(generi_musicali, 1) >= 2
)
AND NOT (provenienza = 'Roma')
AND nome ILIKE 'M%';
```

Q3 - QUERY CON ALMENO UN JOIN E ALMENO UNA CONDIZIONE DI SELEZIONE

LINGUAGGIO NATURALE

Tutte le squadre con il nome della lega e dell'artista capitano

SQL

```
SELECT
  s.cod_squadra,
  s.nome AS nome_squadra,
  l.nome_lega,
  a.nome AS nome_artista,
  a.cognome AS cognome_artista,
  s.punteggio_totale
FROM squadra s
JOIN lega l
  ON s.nome_lega = l.nome_lega
JOIN artista a
  ON s.artista_cf = a.cf
WHERE s.punteggio_totale > 100
ORDER BY s.punteggio_totale DESC;
```

1D - PROGETTO FISICO

[Riportare nella seguente tabella l'elenco degli indici che si intendono creare per: (1) ciascuna query del carico di lavoro individualmente; (2) l'insieme delle query del carico di lavoro, motivando opportunamente, in modo sintetico, le scelte effettuate]

Id query	Relazione	Chiave di ricerca	Tipo (ordinato/hash, clusterizzato/non clusterizzato)	Motivazione
Q1	artista	prezzo (ordinabile in DESC)	BTREE	Per WHERE prezzo ≥ 50 + ORDER BY prezzo DESC LIMIT 10.
Q2	artista	lower(nome) text_pattern_ops	BTREE	Per ricerche di prefisso tipo lower(nome) LIKE 'm%'.
Q3	squadra	punteggio_totale (DESC)	BTREE	Per WHERE punteggio_totale > 100 + ordinamento.

Schema fisico complessivo per il carico di lavoro	Motivazione
Creo 3 indici BTREE (niente clustering/partizioni): artista(prezzo DESC), artista(lower(nome) text_pattern_ops), squadra(punteggio_totale DESC). Dopo la creazione eseguo ANALYZE.	• Q1 → artista(prezzo DESC): filtra su prezzo ≥ ... e ordina in DESC senza fare sort (Top-N). • Q2 → artista(lower(nome) text_pattern_ops): ricerche di prefisso tipo m% (uso lower(nome) LIKE 'm%'). • Q3 → squadra(punteggio_totale DESC): velocizza WHERE > 100 e l'ORDER BY decrescente. Scelgo pochi indici per tenere il progetto semplice e non rallentare le scritture.

Q1 - QUERY CON SINGOLA SELEZIONE E NESSUN JOIN

PIANO DI ESECUZIONE SCELTO DAL SISTEMA PRIMA DELLA CREAZIONE DELLO SCHEMA FISICO

Limit (cost=0.28..1.69 rows=10 width=29) (actual time=0.026..0.066 rows=10 loops=1)

" Output: cf, nome, cognome, prezzo"

Buffers: shared hit=12

-> Index Scan using ix_artista_prezzo_desc on fantasanremo.artista (cost=0.28..210.25 rows=1488 width=29) (actual time=0.025..0.062 rows=10 loops=1)

" Output: cf, nome, cognome, prezzo"

Index Cond: (artista.prezzo >= 50)

Buffers: shared hit=12

Planning:

Buffers: shared hit=12

Planning Time: 0.243 ms

Execution Time: 0.087 ms

PIANO DI ESECUZIONE SCELTO DAL SISTEMA DOPO DELLA CREAZIONE DELLO SCHEMA FISICO

Limit (cost=0.28..1.69 rows=10 width=29) (actual time=0.015..0.026 rows=10 loops=1)

" Output: cf, nome, cognome, prezzo"

Buffers: shared hit=12

-> Index Scan using ix_artista_prezzo_desc on fantasanremo.artista (cost=0.28..210.25 rows=1488 width=29) (actual time=0.014..0.023 rows=10 loops=1)

" Output: cf, nome, cognome, prezzo"

Index Cond: (artista.prezzo >= 50)

Buffers: shared hit=12

Planning:

Buffers: shared hit=12

Planning Time: 0.175 ms

Execution Time: 0.040 ms

CONFRONTO TRA I DUE PIANI

Tempo esecuzione, PRIMA	Tempo esecuzione DOPO	Motivazione
0.087 ms	0.040 ms	Prima (senza indice): il database deve leggere tante righe, poi metterle in ordine per prezzo e infine prendere le prime 10. Quindi fa più lavoro e impiega più tempo. Dopo (con indice): l'indice è già in ordine DESC, applica prezzo ≥ ..., prende subito le prime 10 e non fa sort di conseguenza si hanno meno letture

Q2 - QUERY CON CONDIZIONE DI SELEZIONE COMPLESSA E NESSUN JOIN

PIANO DI ESECUZIONE SCELTO DAL SISTEMA PRIMA DELLA CREAZIONE DELLO SCHEMA FISICO

Index Scan using ix_artista_lower_nome_pat on fantasanremo.artista (cost=0.28..8.31 rows=1 width=63)
(actual time=0.043..0.043 rows=0 loops=1)
" Output: cf, nome, cognome, prezzo, generi_musicali, provenienza"
Index Cond: ((lower((artista.nome)::text) ~>= ~ 'm'::text) AND (lower((artista.nome)::text) ~<~ 'n'::text))
" Filter: (((artista.provenienza)::text <> 'Roma'::text) AND (lower((artista.nome)::text) ~~ 'm%'::text) AND
((artista.prezzo > 50) OR (array_length(artista.generi_musicali, 1) >= 2)))"
Rows Removed by Filter: 1
Buffers: shared hit=3
Planning:
Buffers: shared hit=21
Planning Time: 0.473 ms
Execution Time: 0.076 ms

PIANO DI ESECUZIONE SCELTO DAL SISTEMA DOPO DELLA CREAZIONE DELLO SCHEMA FISICO

Index Scan using ix_artista_lower_nome_pat on fantasanremo.artista (cost=0.28..8.31 rows=1 width=63)
(actual time=0.015..0.016 rows=0 loops=1)
" Output: cf, nome, cognome, prezzo, generi_musicali, provenienza"
Index Cond: ((lower((artista.nome)::text) ~>= ~ 'm'::text) AND (lower((artista.nome)::text) ~<~ 'n'::text))
" Filter: (((artista.provenienza)::text <> 'Roma'::text) AND (lower((artista.nome)::text) ~~ 'm%'::text) AND
((artista.prezzo > 50) OR (array_length(artista.generi_musicali, 1) >= 2)))"
Rows Removed by Filter: 1
Buffers: shared hit=3
Planning:
Buffers: shared hit=21
Planning Time: 0.301 ms
Execution Time: 0.037 ms

CONFRONTO TRA I DUE PIANI

Tempo esecuzione PRIMA	Tempo esecuzione DOPO	Motivazione
0.076 ms	0.037 ms	Senza l'indice su lower(nome) Q2 scorrerebbe tutta la tabella artista riga per riga e per ognuna calcola lower(nome) e controlla se inizia con 'm' per poi applicare: provenienza <> 'Roma' e (prezzo > 50 OR array_length(generi_musicali,1) >= 2). L'indice permette di andare direttamente sui nomi che iniziano per "m"

Q3 - QUERY CON ALMENO UN JOIN E ALMENO UNA CONDIZIONE DI SELEZIONE

PIANO DI ESECUZIONE SCELTO DAL SISTEMA PRIMA DELLA CREAZIONE DELLO SCHEMA FISICO

Sort (cost=273.65..278.40 rows=1900 width=51) (actual time=4.703..4.840 rows=1900 loops=1)
" Output: s.cod_squadra, s.nome, l.nome_lega, a.nome, a.cognome, s.punteggio_totale"
Sort Key: s.punteggio_totale DESC
Sort Method: quicksort Memory: 182kB
Buffers: shared hit=63
-> Hash Join (cost=117.07..170.18 rows=1900 width=51) (actual time=1.843..3.775 rows=1900 loops=1)
" Output: s.cod_squadra, s.nome, l.nome_lega, a.nome, a.cognome, s.punteggio_totale"
Inner Unique: true
Hash Cond: ((s.artista_cf)::text = (a.cf)::text)
Buffers: shared hit=63
-> Hash Join (cost=6.52..54.64 rows=1900 width=38) (actual time=0.157..1.224 rows=1900 loops=1)
" Output: s.cod_squadra, s.nome, s.punteggio_totale, s.artista_cf, l.nome_lega"
Inner Unique: true
Hash Cond: ((s.nome_lega)::text = (l.nome_lega)::text)
Buffers: shared hit=20
-> Seq Scan on fantasanremo.squadra s (cost=0.00..43.03 rows=1900 width=38) (actual time=0.052..0.379 rows=1900 loops=1)
" Output: s.cod_squadra, s.nome, s.punteggio_totale, s.nome_lega, s.artista_cf"
Filter: (s.punteggio_totale > 100)
Rows Removed by Filter: 102
Buffers: shared hit=18
-> Hash (cost=4.01..4.01 rows=201 width=8) (actual time=0.097..0.098 rows=201 loops=1)
Output: l.nome_lega
Buckets: 1024 Batches: 1 Memory Usage: 16kB
Buffers: shared hit=2
-> Seq Scan on fantasanremo.lega l (cost=0.00..4.01 rows=201 width=8) (actual time=0.013..0.038 rows=201 loops=1)
Output: l.nome_lega
Buffers: shared hit=2
-> Hash (cost=73.02..73.02 rows=3002 width=25) (actual time=1.658..1.658 rows=3002 loops=1)
" Output: a.nome, a.cognome, a.cf"
Buckets: 4096 Batches: 1 Memory Usage: 205kB
Buffers: shared hit=43
-> Seq Scan on fantasanremo.artista a (cost=0.00..73.02 rows=3002 width=25) (actual time=0.013..0.627 rows=3002 loops=1)
" Output: a.nome, a.cognome, a.cf"
Buffers: shared hit=43
Planning:
Buffers: shared hit=25
Planning Time: 1.390 ms
Execution Time: 5.111 ms

PIANO DI ESECUZIONE SCELTO DAL SISTEMA DOPO DELLA CREAZIONE DELLO SCHEMA FISICO

Sort (cost=273.65..278.40 rows=1900 width=51) (actual time=2.761..2.839 rows=1900 loops=1)
" Output: s.cod_squadra, s.nome, l.nome_lega, a.nome, a.cognome, s.punteggio_totale"
Sort Key: s.punteggio_totale DESC
Sort Method: quicksort Memory: 182kB
Buffers: shared hit=63
-> Hash Join (cost=117.07..170.18 rows=1900 width=51) (actual time=0.861..2.121 rows=1900 loops=1)

```

"    Output: s.cod_squadra, s.nome, l.nome_lega, a.nome, a.cognome, s.punteggio_totale"
    Inner Unique: true
    Hash Cond: ((s.artista_cf)::text = (a.cf)::text)
    Buffers: shared hit=63
    -> Hash Join (cost=6.52..54.64 rows=1900 width=38) (actual time=0.077..0.883 rows=1900 loops=1)
"    Output: s.cod_squadra, s.nome, s.punteggio_totale, s.artista_cf, l.nome_lega"
    Inner Unique: true
    Hash Cond: ((s.nome_lega)::text = (l.nome_lega)::text)
    Buffers: shared hit=20
    -> Seq Scan on fantasanremo.squadra s (cost=0.00..43.03 rows=1900 width=38) (actual
time=0.021..0.200 rows=1900 loops=1)
"    Output: s.cod_squadra, s.nome, s.punteggio_totale, s.nome_lega, s.artista_cf"
    Filter: (s.punteggio_totale > 100)
    Rows Removed by Filter: 102
    Buffers: shared hit=18
    -> Hash (cost=4.01..4.01 rows=201 width=8) (actual time=0.051..0.052 rows=201 loops=1)
    Output: l.nome_lega
    Buckets: 1024 Batches: 1 Memory Usage: 16kB
    Buffers: shared hit=2
    -> Seq Scan on fantasanremo.lega l (cost=0.00..4.01 rows=201 width=8) (actual
time=0.007..0.021 rows=201 loops=1)
    Output: l.nome_lega
    Buffers: shared hit=2
    -> Hash (cost=73.02..73.02 rows=3002 width=25) (actual time=0.767..0.768 rows=3002 loops=1)
"    Output: a.nome, a.cognome, a.cf"
    Buckets: 4096 Batches: 1 Memory Usage: 205kB
    Buffers: shared hit=43
    -> Seq Scan on fantasanremo.artista a (cost=0.00..73.02 rows=3002 width=25) (actual
time=0.006..0.292 rows=3002 loops=1)
"    Output: a.nome, a.cognome, a.cf"
    Buffers: shared hit=43
Planning:
    Buffers: shared hit=25
Planning Time: 0.441 ms
Execution Time: 2.998 ms

```

CONFRONTO TRA I DUE PIANI

Tempo esecuzione PRIMA	Tempo esecuzione DOPO	Motivazione
5.111 ms	2.998 ms	Tiene i punteggi già ordinati dal più alto al più basso. Quando chiedo "le prime N squadre" il database può prendere subito le più alte e fermarsi presto nonostante questo rimane un indice poco efficiente perchè prende quasi tutte le righe nella query

2. CONTROLLO DELL'ACCESSO

GERARCHIA TRA I RUOLI

GERARCHIA

Amministratore FantaSanremo → Proprietario lega → Amministratore lega → Utente premium.

MOTIVAZIONE GERARCHIA

L'Amministratore FantaSanremo gestisce tutto lo schema (crea tabelle/indici e può fare tutto).

Il Proprietario lega può creare/gestire la propria lega e i relativi iscritti/squadre.

L'Amministratore lega aiuta a gestire squadre/iscritti della lega.

L'Utente premium usa l'app: vede i dati pubblici e può creare squadra/formazioni.

ASSEGNAZIONE PRIVILEGI SPECIFICI AI RUOLI

[Riportare nella prima colonna della seguente tabella le relazioni considerate e in ciascuna altra cella (i,j) i privilegi specifici (quindi non acquisiti tramite la gerarchia) che si intendono assegnare al ruolo j sulla tabella i].

RELAZIONE	Amministratore del FantaSanremo	Utente premium	Amministratore lega	Proprietario lega
artista, brano, serata, esibizione, effettua,	ALL	SELECT	SELECT	SELECT
lega	ALL	SELECT	SELECT	INSERT/UPDATE/DELETE
iscritto	ALL	INSERT	INSERT/UPDATE/DELETE	INSERT/UPDATE/DELETE
squadra	ALL	INSERT	INSERT/UPDATE/DELETE	INSERT/UPDATE/DELETE
formazione	ALL	INSERT	INSERT/UPDATE/DELETE	INSERT/UPDATE/DELETE